
Tribhuvan University
Faculty of Humanities & Social Sciences
OFFICE OF THE DEAN

Web Technology

BCA — Semester 3 — 2019 — Answer Sheet

Subject Code: CACS205

Group A

Attempt all. [10×1=10]

1. i) Why we use XSLT language?

a) Use to delete XML nodes b) Use to rename XML nodes c) Use to transform XML documents d) Use to sort XML nodes

ii) Which is the correct CSS syntax?

a) `body {color: black}` b) `body:color=black` c) `{body;color:black}` d) `{body=color:black}`

iii) SQL Query to delete all rows in a table without deleting the table structure?

a) `DELETE FROM table_name;` b) `DROP TABLE table_name;` c) `REMOVE FROM table_name;` d) `TRUNCATE table_name;`

iv) Web design that makes your web page look good on all devices?

a) Static web design b) Dynamic web design c) Responsive web design d) Adaptive web design

v) Constraint on the data per cookie?

a) 1 KB b) 2 KB c) 4 KB d) 8 KB

vi) Which is not an inline element?

a) `<span>` b) `<a>` c) `<img>` d) `<div>`

vii) Which is wrong in XQuery?

a) Used to query XML data b) Built on XPath expressions c) Used to generate tables for XSLT d) Used to extract and manipulate data from XML

viii) Which method has much larger limit on amount of data that can be passed to web server?

a) Get method b) Post method c) Put method d) Delete method

ix) In HTML form `<input type="text">` is used for?

a) One-line text b) Multi-line text c) Password field d) Hidden field

x) Which inline function embeds an independent HTML document into current document?

a) `<frame>` b) `<object>` c) `<iframe>` d) `<embed>`

[10 × 1 = 10]

Answer: Answers:

i) c) Use to transform XML documents — XSLT (Extensible Stylesheet Language Transformations) is a language used to transform XML documents into other formats such as HTML, plain text, or another XML document. It uses XPath to navigate XML documents and define transformation rules.

ii) a) `body {color: black}` — CSS syntax follows the pattern: `selector {property: value;}`. The correct syntax is `body {color: black;}` where 'body' is the selector, 'color' is the property, and 'black' is the value.

iii) a) `DELETE FROM table_name;` — The `DELETE` statement removes all rows from a table while preserving the table structure, constraints, and indexes. `TRUNCATE` also removes all rows but is a DDL command. `DROP TABLE` removes the entire table including its structure.

iv) c) Responsive web design — Responsive web design uses CSS media queries, flexible grids, and fluid images to automatically adjust the layout and content based on the screen size and orientation of the device (mobile, tablet, desktop).

v) c) 4 KB — Cookies are limited to approximately 4 KB (4096 bytes) of data per cookie. Browsers also limit the total number of cookies per domain (typically around 20-50 cookies).

vi) d) <div> — <div> is a block-level element that takes up the full width available and starts on a new line. , <a>, and are all inline elements that flow within text without breaking the line.

vii) c) Used to generate tables for XSLT — This statement is wrong. XQuery is a query language for XML (like SQL for databases), built on XPath expressions. It is used to query, extract, and manipulate data from XML documents. XSLT is the language used for transforming XML, not XQuery.

viii) b) Post method — The POST method sends data in the HTTP request body, allowing much larger amounts of data to be transmitted compared to GET, which appends data to the URL (limited by URL length, typically ~2048 characters). POST is also more secure for sensitive data.

ix) a) One-line text — The <input type="text"> creates a single-line text input field where users can enter short text. For multi-line text, the <textarea> element is used.

x) c) <iframe> — The <iframe> (inline frame) element embeds an independent HTML document within the current document. It is commonly used to embed external content such as maps, videos, or other web pages.

Group B

Attempt any SIX. [6×5=30]

2. What is HTML Link? Explain different links used in HTML.

[1+4]

Answer: HTML Link:

An HTML link (or hyperlink) is a reference to another resource on the web. It allows users to navigate from one page to another, download files, send emails, or jump to specific sections within the same page. Links are created using the `<a>` (anchor) tag with the **href** attribute.

Syntax: `<a href="url">Link Text</a>`

Different Types of Links in HTML:

1. External Link (Absolute URL): Links to a page on another website.

`<a href="https://www.example.com">Visit Example</a>`

2. Internal Link (Relative URL): Links to a page within the same website.

`<a href="about.html">About Us</a>`

`<a href="/contact.html">Contact</a>`

3. Anchor Link (Bookmark): Links to a specific section within the same page using id.

`<a href="#section1">Go to Section 1</a>`

`<h2 id="section1">Section 1</h2>`

4. Email Link: Opens the default email client with a pre-filled address.

`<a href="mailto:someone@example.com">Send Email</a>`

5. Download Link: Prompts the user to download a file.

`<a href="document.pdf" download>Download PDF</a>`

6. Telephone Link: Initiates a phone call on mobile devices.

`<a href="tel:+9771234567890">Call Us</a>`

7. Target Attribute: Controls where the linked document opens.

`<a href="https://example.com" target="_blank">Open in New Tab</a>`

3. Write HTML tags to generate a table with columns: Routine, Sunday, Monday, Break, Tuesday and rows: WT, DSA, SAD, Java, Statistics, Math.

[5]

Answer: HTML Code for Class Routine Table:

```
<!DOCTYPE html> <html> <head> <title>Class Routine</title> <style> table { border-collapse: collapse; width: 80%; margin: auto; } th, td { border: 1px solid black; padding: 10px; text-align: center; } th { background-color: #4CAF50; color: white; } tr:nth-child(even) { background-color: #f2f2f2; } </style> </head> <body> <table> <tr> <th>Routine</th> <th>Sunday</th> <th>Monday</th> <th>Break</th> <th>Tuesday</th> </tr> <tr> <td>WT</td> <td>DSA</td> <td>SAD</td> <td>-</td> <td>Java</td> </tr> <tr> <td>Statistics</td> <td>Math</td> <td>-</td> <td>-</td> <td>WT</td> </tr> </table> </body> </html>
```

4. Write HTML tags to generate nested list output: FOHSS (BCA -> IoT, BSW), FOM (BIM, BBA).

[5]

Answer: HTML Code for Nested List:

```
<!DOCTYPE html> <html> <head> <title>Nested List</title> </head> <body> <ul> <li>FOHSS <ul> <li>BCA
<ul> <li>IoT</li> <li>BSW</li> </ul> </li> </ul> </li> <li>FOM <ul> <li>BIM</li> <li>BBA</li> </ul> </li> </ul>
</body> </html>
```

Output:

- FOHSS
 - BCA
 - IoT
 - BSW
- FOM
 - BIM
 - BBA

5. "It is discouraged to use frame in web development" Explain the reasons.

[5]

Answer: Reasons Why Frames Are Discouraged in Web Development:

1. Deprecated in HTML5:

The <frame> and <frameset> tags have been removed from the HTML5 specification. Modern browsers still support them for backward compatibility, but they are considered obsolete. The <iframe> element is the recommended alternative for embedding content.

2. Poor Accessibility:

Screen readers and assistive technologies struggle with frames because they treat each frame as a separate document. Users with disabilities may find it difficult to navigate between frames and understand the overall page structure.

3. SEO (Search Engine Optimization) Issues:

Search engines have difficulty indexing content within frames. Each frame loads a separate HTML document, making it hard for search engines to understand the relationship between the content and to index the complete page properly. This negatively impacts search rankings.

4. Bookmarking and URL Problems:

When a page uses frames, the browser's address bar shows only the URL of the frameset document, not the individual frames. Users cannot bookmark or share specific content within a frame, and the back/forward navigation buttons may behave unexpectedly.

5. Mobile Responsiveness:

Frames are not responsive by design. They do not adapt well to different screen sizes, making websites with frames unusable on mobile devices and tablets. Modern responsive design techniques (CSS Flexbox, Grid) provide much better solutions.

6. Maintenance Difficulty:

Websites built with frames are harder to maintain and update. Navigation menus embedded in separate frames require changes across multiple files. Modern templating systems and server-side includes offer better alternatives.

7. Usability Issues:

Frames create confusing user experiences. Users cannot easily print the entire page, scroll behavior becomes inconsistent across frames, and the page may display differently across browsers.

6. What is the strength of CSS? What are the various approaches to include CSS document in HTML?

[1+4]

Answer: Strength of CSS (Cascading Style Sheets):

- 1. Separation of Content and Presentation:** CSS separates the structure (HTML) from the styling, making code cleaner and easier to maintain.
- 2. Reusability:** A single CSS file can style multiple HTML pages, ensuring consistency across an entire website.
- 3. Consistent Design:** Changes made to the CSS file are automatically applied to all linked pages, ensuring uniform appearance.
- 4. Faster Page Loading:** External CSS files are cached by browsers, reducing load times for subsequent pages.
- 5. Responsive Design:** CSS media queries enable websites to adapt to different screen sizes and devices.
- 6. Enhanced Accessibility:** CSS allows developers to create accessible designs that work with screen readers and assistive technologies.
- 7. Print-Friendly Pages:** CSS can define separate styles for printing, hiding unnecessary elements and optimizing layout.
- 8. Advanced Visual Effects:** CSS supports animations, transitions, gradients, shadows, and transforms without requiring JavaScript or images.

Various Approaches to Include CSS in HTML:

- 1. Inline CSS:** Styles are applied directly to individual HTML elements using the style attribute.

```
<p style="color: blue; font-size: 16px;">This is a blue paragraph.</p>
```

- 2. Internal (Embedded) CSS:** Styles are defined within the <style> tag in the <head> section of the HTML document.

```
<head>
  <style>
    body { background-color: lightgray; }
    h1 { color: navy; }
  </style>
</head>
```

- 3. External CSS:** Styles are written in a separate .css file and linked to the HTML document using the <link> tag. This is the recommended approach.

```
<head>
  <link rel="stylesheet" type="text/css" href="styles.css">
</head>
```

- 4. @import Rule:** CSS files can import other CSS files using the @import rule within a <style> tag or external CSS file.

```
<style>
  @import url("external.css");
</style>
```

7. What is cookie? Explain how you set and remove values stored in cookie with suitable example.

[1+4]

Answer: Cookie:

A cookie is a small piece of data (text file) stored on the user's computer by a web browser at the request of a web server. Cookies are used to remember user preferences, login sessions, shopping cart items, and track user behavior. Each cookie has a name, value, expiration date, domain, and path. The maximum size of a cookie is approximately 4 KB.

Setting and Removing Cookies:

Cookies can be created, read, and deleted using JavaScript or server-side scripts. Here is an example using JavaScript:

```
<!DOCTYPE html> <html> <head> <title>Cookie Example</title> </head> <body> <h2>Cookie Demo</h2>
<button onclick="setCookie()">Set Cookie</button> <button onclick="getCookie()">Get Cookie</button>
<button onclick="removeCookie()">Remove Cookie</button> <p id="output"></p> <script> // Function to set a
cookie function setCookie() { var username = "JohnDoe"; var expires = new Date();
expires.setTime(expires.getTime() + (7 * 24 * 60 * 60 * 1000)); // 7 days document.cookie = "username=" +
username + ";expires=" + expires.toUTCString() + ";path=/"; document.getElementById("output").innerHTML =
"Cookie set successfully!"; } // Function to get a cookie function getCookie() { var name = "username="; var
decodedCookie = decodeURIComponent(document.cookie); var cookieArray = decodedCookie.split(';');
for(var i = 0; i < cookieArray.length; i++) { var c = cookieArray[i].trim(); if (c.indexOf(name) == 0) {
document.getElementById("output").innerHTML = "Cookie value: " + c.substring(name.length, c.length);
return; } } document.getElementById("output").innerHTML = "Cookie not found!"; } // Function to remove/delete
a cookie function removeCookie() { document.cookie = "username=;expires=Thu, 01 Jan 1970 00:00:00
UTC;path=/"; document.getElementById("output").innerHTML = "Cookie removed successfully!"; } </script>
</body> </html>
```

Explanation:

Setting a Cookie: Use **document.cookie** with name=value pair, expiration date, and path. The expiration date determines how long the cookie persists.

Reading a Cookie: Access **document.cookie** which returns all cookies as a semicolon-separated string. Parse the string to extract the desired cookie value.

Removing a Cookie: Set the cookie's expiration date to a past date (e.g., January 1, 1970). The browser automatically deletes expired cookies.

8. Write HTML tags to generate a form with: Student Information, First Name, Last Name, Gender (radio), Subject (checkboxes: C, JAVA, PHP), Comment (textarea).

[5]

Answer: HTML Code for Student Information Form:

```
<!DOCTYPE html> <html> <head> <title>Student Information</title> <style> body { font-family: Arial, sans-serif; margin: 20px; } form { width: 400px; padding: 20px; border: 1px solid #ccc; border-radius: 5px; } label { display: inline-block; width: 100px; margin-top: 10px; } input[type="text"], textarea { width: 250px; padding: 5px; margin-top: 5px; } .gender, .subjects { margin-top: 10px; } input[type="submit"] { margin-top: 15px; padding: 8px 20px; background-color: #4CAF50; color: white; border: none; cursor: pointer; } </style> </head> <body> <form action="/submit" method="post"> <h2>Student Information</h2> <label for="fname">First Name:</label> <input type="text" id="fname" name="firstname" placeholder="Enter first name"><br/> <label for="lname">Last Name:</label> <input type="text" id="lname" name="lastname" placeholder="Enter last name"><br/> <div class="gender"> <label>Gender:</label> <input type="radio" id="male" name="gender" value="male"> <label for="male">Male</label> <input type="radio" id="female" name="gender" value="female"> <label for="female">Female</label> <input type="radio" id="other" name="gender" value="other"> <label for="other">Other</label> </div> <div class="subjects"> <label>Subject:</label><br/> <input type="checkbox" id="c" name="subject" value="C"> <label for="c">C</label><br/> <input type="checkbox" id="java" name="subject" value="JAVA"> <label for="java">JAVA</label><br/> <input type="checkbox" id="php" name="subject" value="PHP"> <label for="php">PHP</label> </div> <label for="comment">Comment:</label><br/> <textarea id="comment" name="comment" rows="4" cols="35" placeholder="Enter your comments here..."></textarea><br/> <input type="submit" value="Submit"> </form> </body> </html>
```

Group C

Attempt any TWO. [2×10=20]

9. Is XML replacement for HTML? Explain different types of tier technology with suitable diagram.

[2+8]

Answer: Is XML a Replacement for HTML?

No, XML is NOT a replacement for HTML. XML and HTML were designed with different goals and serve different purposes:

HTML (HyperText Markup Language):

- Designed to display data and focus on how data looks.
- Predefined tags (<p>, <div>, <table>, etc.) that browsers understand.
- Used to create web pages and define their structure and presentation.
- Tags are primarily for formatting and layout.

XML (Extensible Markup Language):

- Designed to carry and store data, focusing on what data is.
- No predefined tags; users create their own tags to describe data.
- Used for data interchange between systems and platforms.
- Self-descriptive and platform-independent.

Relationship:

XML complements HTML. XML is used to transport and store data, while HTML is used to format and display data. XHTML combines both by using XML syntax rules with HTML tags.

Three-Tier Architecture:

Three-tier architecture is a client-server architecture that separates an application into three logical and physical computing tiers: Presentation Tier, Application Tier, and Data Tier.

1. Presentation Tier (Client Tier):

This is the top-most level of the application. It is the user interface and communication layer where the end-user interacts with the application. It displays information to the user and collects input.

Technologies: HTML, CSS, JavaScript, React, Angular

Function: Display data, handle user input, send requests to application tier.

2. Application Tier (Logic/Business Tier):

This middle tier coordinates the application, processes commands, makes logical decisions, and performs calculations. It acts as a bridge between the presentation and data tiers.

Technologies: PHP, Python, Java, Node.js, ASP.NET

Function: Process business logic, validate data, authenticate users, manage sessions, communicate with database.

3. Data Tier (Database Tier):

This bottom tier consists of database servers where information is stored and retrieved. It maintains data integrity and handles data persistence.

Technologies: MySQL, PostgreSQL, MongoDB, Oracle, SQL Server

Function: Store data, execute queries, manage transactions, ensure data security.

Diagram:

+-----+ HTTP Request +-----+ SQL Query +-----+ | PRESENTATION |

```

-----> | APPLICATION | -----> | DATA TIER | | TIER | | TIER | | (Database) | |
(Client/Browser) | <----- | (Web Server/App) | <----- | (MySQL/Oracle) | | HTML, CSS, JS |
HTTP Response | PHP/Java/Node | Query Result | | +-----+ +-----+ +-----+ | | |
| | User Interface Business Logic Data Storage

```

Advantages of Three-Tier Architecture:

- **Scalability:** Each tier can be scaled independently.
- **Maintainability:** Changes in one tier do not affect others.
- **Security:** Data tier is protected behind the application tier.
- **Reusability:** Business logic can be reused across different interfaces.
- **Flexibility:** Different technologies can be used for each tier.

10. Write a server-side script to display all the records stored in student table which resides under database called TU. The structure of student table is (id, name, address).

[10]

Answer: Server-Side Script using PHP with MySQLi:

The following PHP script connects to the MySQL database named "TU", retrieves all records from the "student" table, and displays them in an HTML table format.

```
<!DOCTYPE html> <html> <head> <title>Student Records - TU Database</title> <style> table {
border-collapse: collapse; width: 60%; margin: 20px auto; } th, td { border: 1px solid #333; padding: 10px;
text-align: left; } th { background-color: #4CAF50; color: white; } tr:nth-child(even) { background-color: #f2f2f2; }
h2 { text-align: center; color: #333; } .error { color: red; text-align: center; } .success { color: green; text-align:
center; } </style> </head> <body> <h2>Student Records from TU Database</h2> <?php // Database
configuration $servername = "localhost"; $username = "root"; $password = ""; $database = "TU"; // Create
connection using MySQLi $conn = new mysqli($servername, $username, $password, $database); // Check
connection if ($conn->connect_error) { die("<p class='error'>Connection failed: " . $conn->connect_error .
"</p>"); } // SQL query to select all records from student table $sql = "SELECT id, name, address FROM
student"; $result = $conn->query($sql); // Check if records exist if ($result->num_rows > 0) { echo "<table>";
echo "<tr><th>ID</th><th>Name</th><th>Address</th></tr>"; // Output data of each row while($row =
$result->fetch_assoc()) { echo "<tr>"; echo "<td>" . $row["id"] . "</td>"; echo "<td>" . $row["name"] . "</td>";
echo "<td>" . $row["address"] . "</td>"; echo "</tr>"; } echo "</table>"; echo "<p class='success'>Total
Records: " . $result->num_rows . "</p>"; } else { echo "<p class='error'>No records found in the student
table.</p>"; } // Close connection $conn->close(); ?> </body> </html>
```

Explanation of the Script:

1. Database Connection:

- `$conn = new mysqli($servername, $username, $password, $database);` creates a new MySQLi connection.
- `connect_error` checks if the connection was successful.

2. Execute Query:

- `$sql = "SELECT id, name, address FROM student";` defines the SQL query.
- `$conn->query($sql)` executes the query and returns a result set.

3. Fetch and Display Data:

- `$result->num_rows` returns the number of rows in the result set.
- `$result->fetch_assoc()` fetches each row as an associative array.
- The while loop iterates through all records and displays them in an HTML table.

4. Close Connection:

- `$conn->close();` closes the database connection to free resources.

Alternative using PDO (PHP Data Objects):

```
<?php try { $pdo = new PDO("mysql:host=localhost;dbname=TU", "root", "");
$stmt = $pdo->setAttribute(PDO::ATTR_ERRMODE, PDO::ERRMODE_EXCEPTION);
$stmt = $pdo->query("SELECT id, name, address FROM student");
$students = $stmt->fetchAll(PDO::FETCH_ASSOC);
foreach ($students as $row) { echo "ID: " . $row["id"] . " | Name: " . $row["name"] . " | Address: " . $row["address"] . "<br/>"; } }
catch (PDOException $e) { echo "Error: " . $e->getMessage(); } ?>
```

11. Write well-formed XML document and validate with the use of XML schema. The XML should store Employee information with: Name (First Name, Last Name), Contact (Mobile, Landline), Address (City, Street, Tole).

[10]

Answer: Well-Formed XML Document (employee.xml):

```
<?xml version="1.0" encoding="UTF-8"?>
<Employee xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xsi:noNamespaceSchemaLocation="employee.xsd">
  <Name>
    <FirstName>Ram</FirstName>
    <LastName>Sharma</LastName>
  </Name>
  <Contact>
    <Mobile>9801234567</Mobile>
    <Landline>01-4433221</Landline>
  </Contact>
  <Address>
    <City>Kathmandu</City>
    <Street>Baneshwor</Street>
    <Tole>Minbhawan</Tole>
  </Address>
</Employee>
```

XML Schema (employee.xsd):

```
<?xml version="1.0" encoding="UTF-8"?>
<xs:schema xmlns:xs="http://www.w3.org/2001/XMLSchema">
  <!-- Definition of Name element -->
  <xs:complexType name="NameType">
    <xs:sequence>
      <xs:element name="FirstName" type="xs:string">
        <xs:annotation>
          <xs:documentation>Employee's first name</xs:documentation>
        </xs:annotation>
      </xs:element>
      <xs:element name="LastName" type="xs:string">
        <xs:annotation>
          <xs:documentation>Employee's last name</xs:documentation>
        </xs:annotation>
      </xs:element>
    </xs:sequence>
  </xs:complexType>
  <!-- Definition of Contact element -->
  <xs:complexType name="ContactType">
    <xs:sequence>
      <xs:element name="Mobile" type="xs:string">
        <xs:annotation>
          <xs:documentation>Employee's mobile number</xs:documentation>
        </xs:annotation>
      </xs:element>
      <xs:element name="Landline" type="xs:string">
        <xs:annotation>
          <xs:documentation>Employee's landline number</xs:documentation>
        </xs:annotation>
      </xs:element>
    </xs:sequence>
  </xs:complexType>
  <!-- Definition of Address element -->
  <xs:complexType name="AddressType">
    <xs:sequence>
      <xs:element name="City" type="xs:string">
        <xs:annotation>
          <xs:documentation>City name</xs:documentation>
        </xs:annotation>
      </xs:element>
      <xs:element name="Street" type="xs:string">
        <xs:annotation>
          <xs:documentation>Street name</xs:documentation>
        </xs:annotation>
      </xs:element>
      <xs:element name="Tole" type="xs:string">
        <xs:annotation>
          <xs:documentation>Tole/locality name</xs:documentation>
        </xs:annotation>
      </xs:element>
    </xs:sequence>
  </xs:complexType>
  <!-- Definition of Employee element -->
  <xs:element name="Employee">
    <xs:complexType>
      <xs:sequence>
        <xs:element name="Name" type="NameType" />
        <xs:element name="Contact" type="ContactType" />
        <xs:element name="Address" type="AddressType" />
      </xs:sequence>
    </xs:complexType>
  </xs:element>
</xs:schema>
```

Explanation:

1. Well-Formed XML Rules Followed:

- XML declaration is present (<?xml version="1.0" encoding="UTF-8"?>).
- All elements are properly nested and closed.
- Element names are case-sensitive and consistently used.
- Attribute values are quoted.
- Single root element (<Employee>) contains all other elements.

2. XML Schema (XSD) Components:

- **xs:schema:** Root element defining the schema namespace.
- **xs:complexType:** Defines complex elements that contain child elements.
- **xs:sequence:** Specifies that child elements must appear in the defined order.
- **xs:element:** Declares elements with their name and data type.
- **xs:string:** Built-in data type for text content.

3. Schema Validation:

- The XML file references the schema using `xsi:noNamespaceSchemaLocation="employee.xsd"`.
- XML validators (like online validators or tools like xmllint) check that:
 - All required elements are present (Name, Contact, Address).
 - Elements appear in the correct order.
 - Data types match the schema definitions.
 - No extra or undefined elements exist.

4. Extended Version with Multiple Employees:

```
<?xml version="1.0" encoding="UTF-8"?> <Employees
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xsi:noNamespaceSchemaLocation="employees.xsd"> <Employee> <Name> <FirstName>Sita</FirstName>
<LastName>Gurung</LastName> </Name> <Contact> <Mobile>9812345678</Mobile>
<Landline>01-5544332</Landline> </Contact> <Address> <City>Pokhara</City> <Street>Lakeside</Street>
<Tole>Baidam</Tole> </Address> </Employee> </Employees>
```

Corresponding Schema for Multiple Employees:

```
<xs:schema xmlns:xs="http://www.w3.org/2001/XMLSchema"> <xs:element name="Employees">
<xs:complexType> <xs:sequence> <xs:element name="Employee" maxOccurs="unbounded">
<xs:complexType> <xs:sequence> <xs:element name="Name"> <xs:complexType> <xs:sequence>
<xs:element name="FirstName" type="xs:string"/> <xs:element name="LastName" type="xs:string"/>
</xs:sequence> </xs:complexType> </xs:element> <xs:element name="Contact"> <xs:complexType>
<xs:sequence> <xs:element name="Mobile" type="xs:string"/> <xs:element name="Landline"
type="xs:string"/> </xs:sequence> </xs:complexType> </xs:element> <xs:element name="Address">
<xs:complexType> <xs:sequence> <xs:element name="City" type="xs:string"/> <xs:element name="Street"
type="xs:string"/> <xs:element name="Tole" type="xs:string"/> </xs:sequence> </xs:complexType>
</xs:element> </xs:sequence> </xs:complexType> </xs:element> </xs:sequence> </xs:complexType>
</xs:element> </xs:schema>
```

Note: These answers are prepared for educational purposes. Students are encouraged to verify with official textbooks and course materials.